

numberAI: Building an Image Classifier from Scratch

Project Documentation

May 9, 2026

Abstract

This document outlines the development, architecture, and findings of the **numberAI** project. The objective of this project is to recreate machine learning functionality from scratch, without relying on pre-trained models or established neural network structures. By utilizing simple matrix multiplication and a trial-and-error training approach, the project aims to categorize number images. Additionally, it details the planned second phase involving quantization and distillation for deployment on resource-constrained embedded systems such as Arduino.

Contents

1	Introduction	2
1.1	Project Motivation	2
1.2	Goals and Phases	2
2	System Architecture	2
2.1	Data Processing Pipeline	2
2.2	Model Structure (Hidden Layers)	3
3	Methodology	3
3.1	Inference (Forward Pass)	3
3.2	Training via Trial and Error	3
3.3	Model Reset	3
4	Architecture Iterations	4
4.1	Version 1.0 Architecture	4
4.1.1	Version 1.0 Training Results	4
4.2	Version 2.0 Architecture (Planned)	5
5	Phase 2: Embedded Systems Optimization	5
5.1	Quantization	6
5.2	Distillation	6
6	Discoveries and Challenges	6
6.1	Matrix Size Optimization	6
6.2	Trial and Error Efficiency	6
7	Conclusion	6

1 Introduction

1.1 Project Motivation

Modern machine learning often relies on complex, pre-defined architectures and pre-trained weights. The motivation behind numberAI is to deconstruct this paradigm and build an image classifier from the absolute basics to fully understand the underlying mechanics of neural networks.

1.2 Goals and Phases

The project is divided into two main phases:

- **Phase 1: Foundational Discovery.** To categorize number images using pure matrix multiplication. This phase avoids pre-defined structures, instead relying on trial and error to iteratively discover the optimal matrix size and network topology.
- **Phase 2: Embedded Systems Deployment.** To run the developed classification model on embedded systems with limited resources (e.g., Arduino). This requires applying model compression techniques such as *quantization* and *distillation* to reduce memory footprint and computational load.

2 System Architecture

The system architecture of numberAI is built upon a custom pipeline designed to handle raw images, process them into matrices, apply matrix multiplication, and verify the outputs.

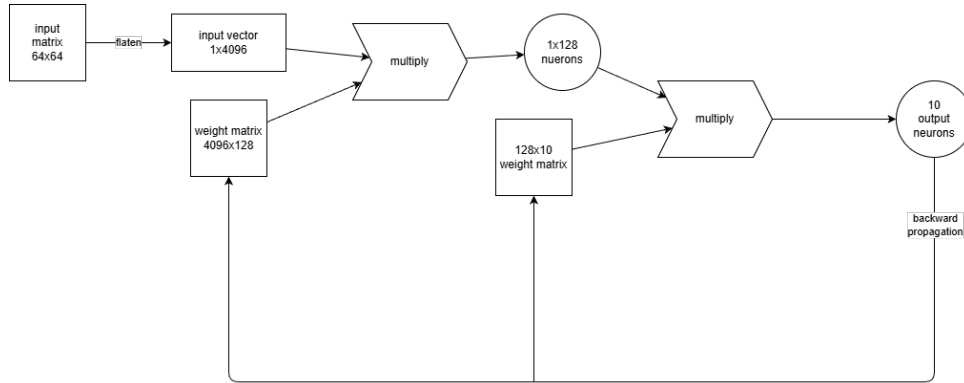


Figure 1: System Architecture of numberAI

2.1 Data Processing Pipeline

The input processing pipeline involves converting raw number images into a format suitable for matrix multiplication. The script `image2txt.py` is responsible for converting input images (stored in `inputImage`) into a text-based matrix representation (stored in `inputText`). These processed inputs are then matched against their corresponding expected outputs (stored in `expectedAnswer`).

2.2 Model Structure (Hidden Layers)

Instead of a fixed neural network topology, the model’s architecture is dynamic. The structure consists of customizable hidden layers whose weights and biases are stored in the `hiddenLayer` directory. The exact dimensions of these matrices are subject to experimentation and tuning during the training phase.

3 Methodology

3.1 Inference (Forward Pass)

The core inference logic is implemented in `run_model.py`. The process relies purely on sequential matrix multiplications of the input text data with the matrices defined in the hidden layers to yield a final categorization prediction.

3.2 Training via Trial and Error

Unlike traditional backpropagation implementations, the training process handled by `train_model.py` employs a numerical gradient estimation approach, often framed as trial-and-error.

The loss function used to evaluate the model’s performance calculates the sum of the Euclidean distances between the actual output vector ($\mathbf{A}$) and the expected output vector ($\mathbf{EA}$) across all 10 training samples:

$$\mathbf{Loss} = \sum_{k=0}^9 \sqrt{\sum_{n=0}^9 (A_{k,n} - EA_{k,n})^2} \quad (1)$$

Where A_k is the length-10 output vector for the k -th input number image, and EA_k is the corresponding expected answer.

To adjust the weights and decrease the loss, the algorithm perturbs a specific weight (e.g., $H1_{m,n}$) by a small *change* and observes the new loss ($\mathbf{Loss}_{o+change}$). The weight is then updated using the numerical gradient approximation:

$$H_{m,n} := H_{m,n} - \left(\frac{\mathbf{Loss}_{o+change} - \mathbf{Loss}_o}{change} \right) \cdot \alpha \quad (2)$$

Where $\mathbf{Loss}_o$ is the original loss, and the learning rate α is scaled proportionally to the loss. This heuristic method bypasses complex calculus by explicitly calculating the finite difference to approximate the derivative.

3.3 Model Reset

To facilitate repeated experimentation, `reset_model.py` provides the functionality to re-initialize the matrix weights using random uniform distributions (`np.random.rand`). This allows the trial-and-error process to start fresh from different initial states.

Upon resetting the two primary hidden layers (H1 of size 256×32 and H2 of size 32×10), the initial Shannon entropy of the randomly generated weights was recorded to establish a baseline before training:

- **H1 Entropy:** 6.633968859734219
- **H2 Entropy:** 6.45341914045563

Tracking this entropy provides insight into the information density and randomness of the network before it adapts through trial-and-error iterations.

4 Architecture Iterations

As the project employs a trial-and-error discovery method, the model architecture is expected to evolve through multiple versions as optimal matrix sizes and structures are found.

4.1 Version 1.0 Architecture

The initial structure (**Version 1.0**) consists of the following components:

- **Input Layer:** Flattened representation of the input image.
- **Hidden Layers:** Two primary weight matrices ($H1$ and $H2$), where dimensions were iteratively tested (e.g., $256 \times 32 \rightarrow 32 \times 10$, and later $4096 \times 128 \rightarrow 128 \times 10$).
- **Output Layer:** Vector of size 10×1 , representing the classification scores for digits 0-9.

4.1.1 Version 1.0 Training Results

After training the Version 1.0 architecture via the numerical gradient approach, a measurable drop in Shannon entropy for the weight matrices was observed:

- **Pre-training Entropy:** $H1 \approx 6.63$, $H2 \approx 6.45$
- **Post-training Entropy:** $H1 \approx 3.05$, $H2 \approx 3.23$

This lower entropy strongly indicates that the trial-and-error process successfully converged on structured patterns within the dataset rather than remaining random. A visualization of the training loss can be seen below:

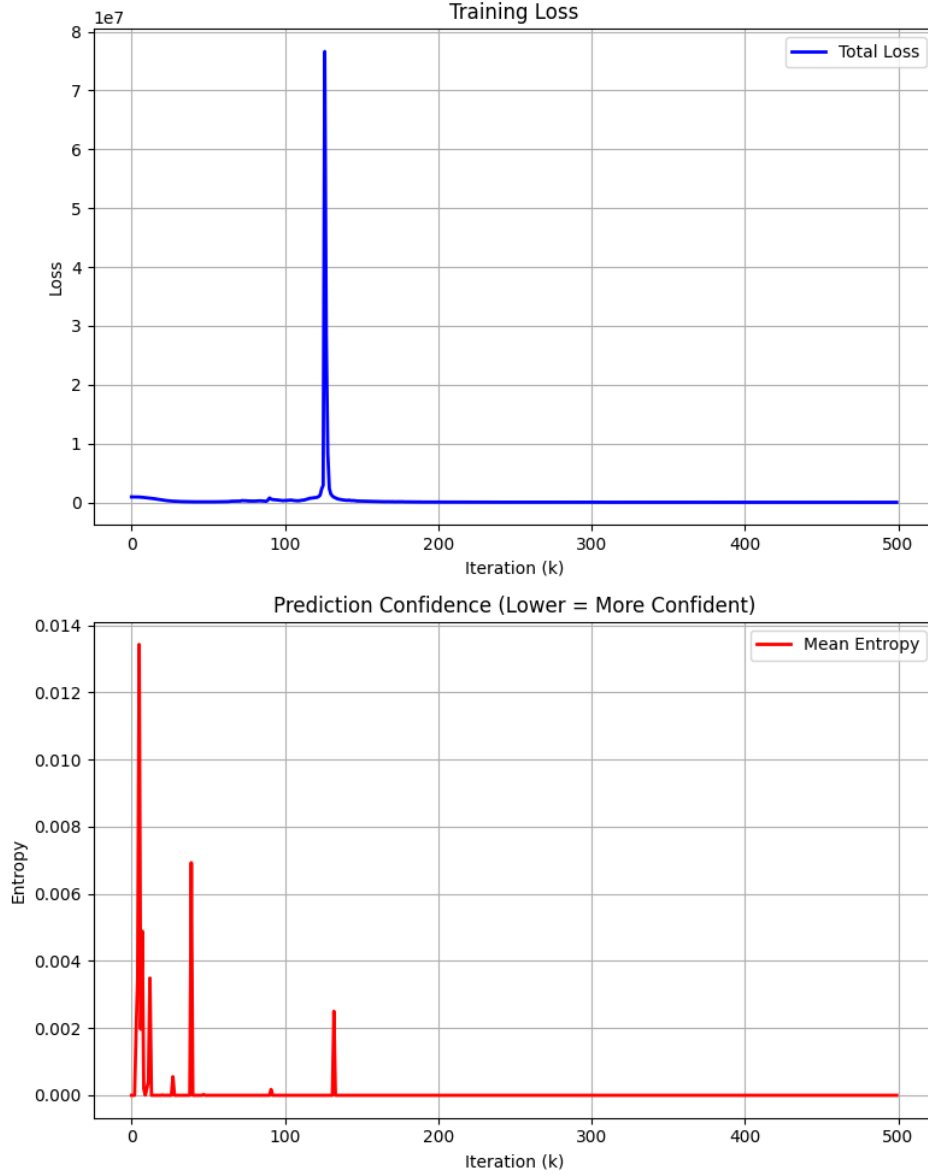


Figure 2: Version 1.0 Training Loss Progression

4.2 Version 2.0 Architecture (Planned)

[Placeholder: Describe changes made in Version 2.0 to improve upon the shortcomings of Version 1.0. For example, adjusting matrix sizes or altering the pipeline to fix accuracy bottlenecks.]

5 Phase 2: Embedded Systems Optimization

To deploy this model on an Arduino, the network must be significantly compressed.

5.1 Quantization

[Placeholder: Describe the process of converting floating-point matrix weights to lower precision formats (e.g., 8-bit integers) and the resulting impact on accuracy.]

5.2 Distillation

[Placeholder: Describe how a smaller, more efficient model (student) is trained to replicate the behavior of the larger, successful trial-and-error model (teacher).]

6 Discoveries and Challenges

6.1 Matrix Size Optimization

[Placeholder: Discuss which matrix dimensions yielded the best balance between accuracy and computational complexity.]

6.2 Trial and Error Efficiency

[Placeholder: Discuss the challenges faced when training without traditional gradient descent. What heuristics were discovered to speed up convergence?]

7 Conclusion

[Placeholder: Summarize the final outcomes of the project, whether the goals were fully met, and potential future improvements.]